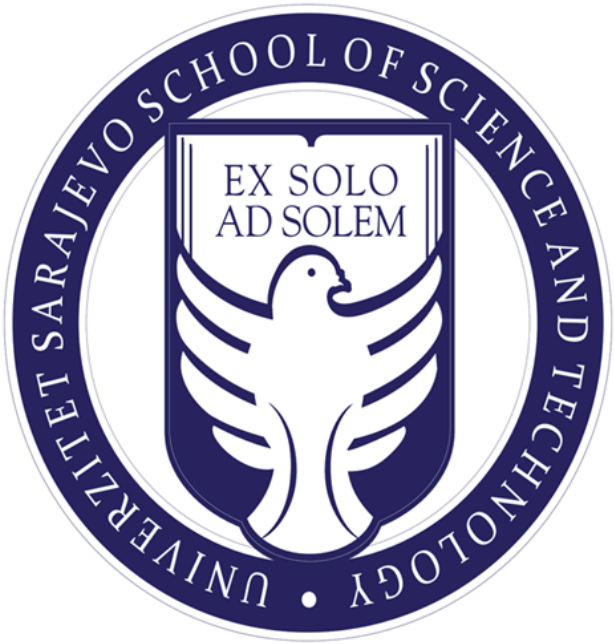
University Sarajevo School of Science and Technology

Faculty of Computer Science



uniPay

Amna Zeljo, Dino Alic, Deni Alic

31st May, 2025

Professor: Prof. Edin Fazlic

[CS380] Software Engineering

Table of Contents

Technical Documentation – Frontend Deployment (Render)..... 3
I. Product: System Documentation..... 3
1. Overview and Background..... 3
2. Architecture and Design Principles..... 3
3. User Story Description..... 3
4. Solution Details..... 4
4.1 GitHub Repository Configuration..... 4
4.2 Render Build and Output Directory Configuration..... 4
4.3 Angular Build Output Confirmation..... 4
4.4 Local Production Build Execution..... 5
4.5 Angular Configuration File (angular.json)..... 5
4.6 Deployed Application Overview in Render..... 5
5. Diagrammatic Representation of the Solution..... 6
6. Milestones..... 6
II. Source Code Documentation..... 7
III. Quality Assurance Documentation..... 7
Technical Documentation – Backend Deployment (DockerHub + Railway)..... 9
I. Product: System Documentation..... 9
1. Overview and Background..... 9
2. Architecture and Design Principles..... 9
3. User Story Description..... 9
4. Solution Details..... 10
4.1 Building the Spring Boot Project with Maven..... 10
4.2 Verifying Application Launch from the JAR..... 10
4.3 Initializing Docker Build with Dockerfile..... 11
4.4 Addressing Docker Tagging and Engine Errors..... 11
4.6 Running the Container Locally and Checking Logs..... 12
4.7 Pushing the Docker Image to DockerHub..... 13
4.8 Configuring Railway Deployment from Docker Image..... 13
4.9 Linking Docker Image to Railway Service..... 14
4.10 Railway Deployment Logs and Active Service..... 14
4.11. Public Networking Configuration on Railway..... 15
4.12. Railway Domain URL for API Access..... 15
4.14. Frontend Integration via environment.prod.ts..... 16
4.15. DevTools Network Tab Verification..... 17
The Finishing Line..... 18
5. Diagrammatic Representation of the Solution..... 19
6. Milestones..... 19
II. Source Code Documentation..... 20
III. Quality Assurance Documentation..... 20
IV. Tools Used for Deployment..... 21
V. Conclusion..... 21

Technical Documentation – Frontend Deployment (Render)

I. Product: System Documentation

---

1. Overview and Background

This document outlines the deployment procedure for the frontend component of the project **uniPay**, developed using the Angular framework. The deployment was conducted using the Render platform for hosting static sites, and the source code is maintained in a public GitHub repository. The objective was to provide a production-ready version of the application accessible via a public URL.

---

2. Architecture and Design Principles

- **Frontend Framework**: Angular
- **Hosting Platform**: Render (Static Site)
- **Build Strategy**: Angular CLI production build
- **Deployment Trigger**: GitHub integration with auto-deploy on push to `main`

The architecture follows a standard static web application pipeline:

[GitHub Repository] → [Render Build Process] → [Public Web Interface]

---

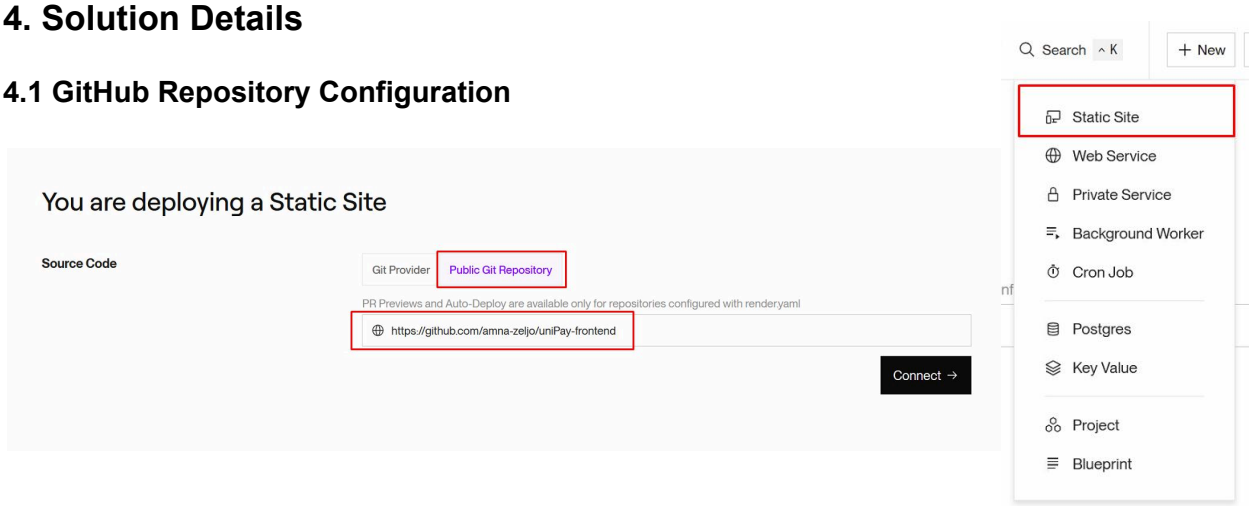
3. User Story Description

As a user, I want to access the uniPay application from any device via a public web link, so that I can interact with the platform without requiring local installation. The application must load the production build correctly and support Angular routing, components, and styling.

---

4. Solution Details

4.1 GitHub Repository Configuration



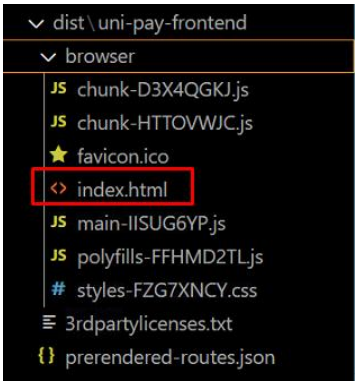
This screenshot displays the structure of the public GitHub repository used for the frontend source code. It confirms that version control is properly configured, and that the repository is integrated with the Render platform for automatic deployment.

4.2 Render Build and Output Directory Configuration



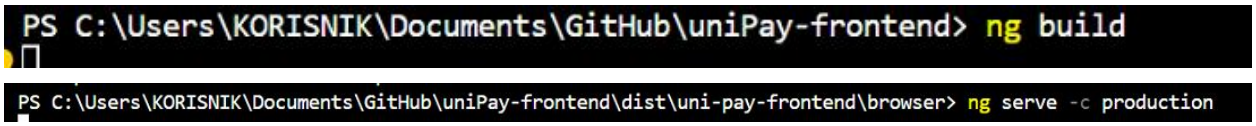
This screenshot presents the Render dashboard with the defined build command (`npm install && ng build -c production`) and the publish directory (`./dist/uni-pay-frontend/browser`). These configurations ensure the Angular application is compiled and served correctly in a production environment.

4.3 Angular Build Output Confirmation



This image confirms the presence of `index.html` and other compiled static files within the output directory (`dist/uni-pay-frontend/browser`). The presence of these files validates the success of the Angular build process.

4.4 Local Production Build Execution



The terminal output demonstrates the successful execution of the Angular build command in production mode. This step ensures that the application is ready for deployment before uploading to the server.

4.5 Angular Configuration File (`angular.json`)



This screenshot shows the relevant section of the `angular.json` file where the production configuration is defined. The output path and environment replacements are explicitly configured to support deployment on Render.

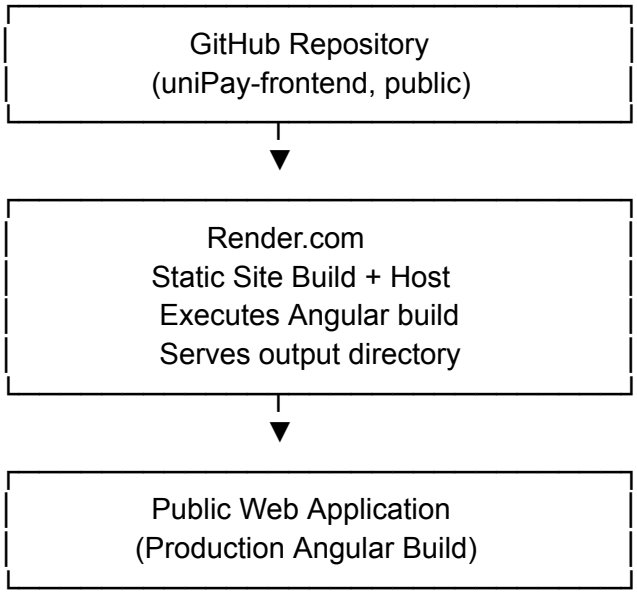
4.6 Deployed Application Overview in Render



This screenshot provides an overview of the deployed frontend application on Render. It includes the site name, deployment status, and production deploy history, verifying that the application is operational and publicly accessible.

---

5. Diagrammatic Representation of the Solution



---

6. Milestones

Task	Status	Completion Date
GitHub repository creation	Completed	28.05.2025
Angular production build generated	Completed	28.05.2025
Static site created on Render	Completed	29.05.2025
GitHub repository connected to Render	Completed	29.05.2025
Application successfully deployed	Completed	29.05.2025

II. Source Code Documentation

- **Repository URL**: `https://github.com/amna-zeljo/uniPay-frontend`
- The repository follows Angular conventions and includes the following:
    - Component modules under `/src/app/components`
    - Services under `/src/app/services`
    - Environment configuration files (`environment.ts`, `environment.prod.ts`)
    - Deployment-specific logic in `angular.json` configuration file

III. Quality Assurance Documentation

Following deployment, the application was tested to ensure:

- The frontend loaded correctly at the deployed Render URL
- No console errors were present during navigation
- Static assets such as stylesheets and scripts were correctly served
- The application responded correctly to user interactions

All tests passed, indicating the deployment is stable and production-ready.

Tools Used for Frontend Deployment

Tool	Purpose
GitHub	Source code repository and version control
Angular CLI	Building production-ready frontend
Render	Hosting static site (frontend)
Visual Studio Code	Code editing and environment management
Terminal (CLI)	Executing Angular build commands

Technical Documentation – Backend Deployment (DockerHub + Railway)

I. Product: System Documentation

1. Overview and Background

This document outlines the deployment procedure of the `uniPay` backend application, developed using Java Spring Boot. The backend exposes RESTful APIs and manages application logic and data processing. For deployment, the backend was containerized using Docker, published to DockerHub, and deployed via Railway using Docker image integration.

The deployment was designed to ensure high portability and seamless integration with cloud infrastructure. By containerizing the backend with Docker, the system is decoupled from the underlying operating system and can run uniformly across different environments.

---

2. Architecture and Design Principles

- **Backend Framework**: Java Spring Boot
- **Containerization**: Docker
- **Hosting Platform**: Railway (Docker image from DockerHub)
- **Build Strategy**: Maven build followed by Docker image generation
- **Deployment Trigger**: Manual Docker image push and Railway redeploy

The backend deployment follows this architecture:

[Local Spring Boot Build] → [Docker Image Build] → [DockerHub Repository] → [Railway Deployment]

---

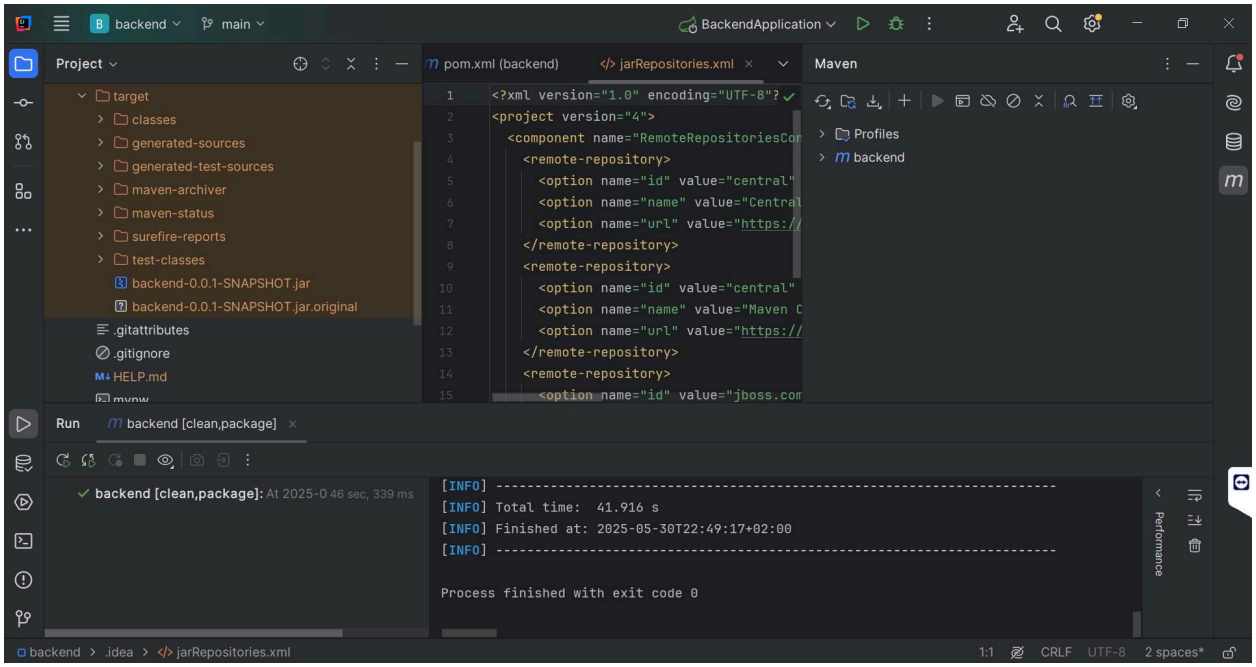
3. User Story Description

As a user, I want to access the backend services of the uniPay application through a reliable REST API endpoint so that the frontend can communicate with it seamlessly.

---

4. Solution Details

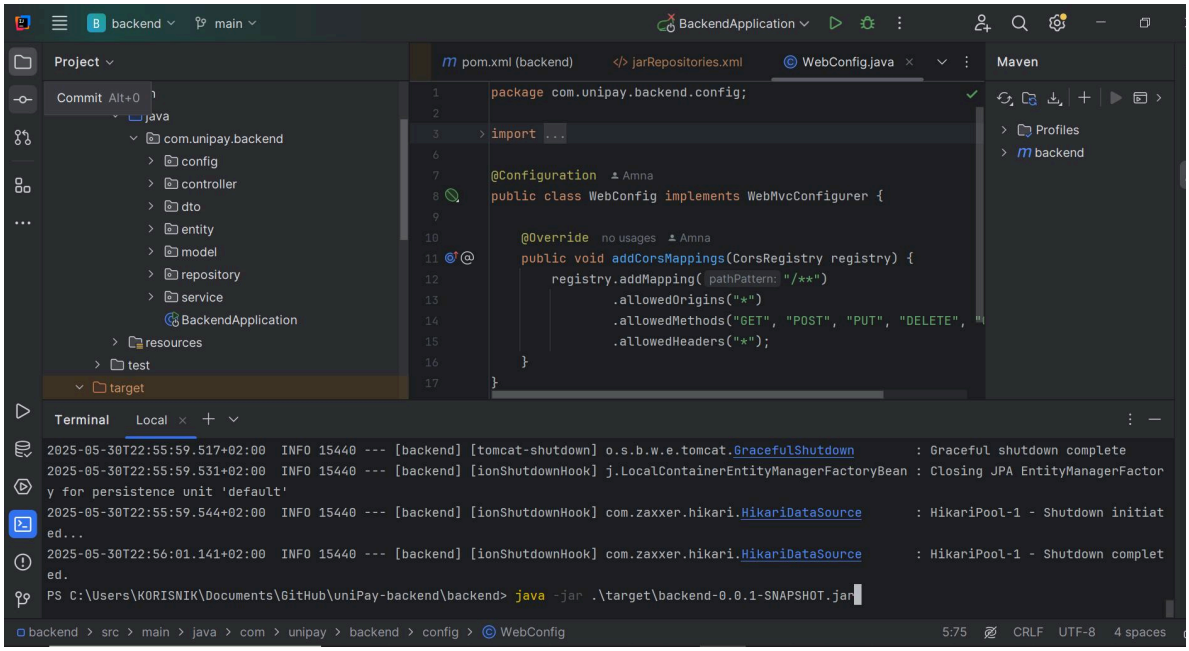
4.1 Building the Spring Boot Project with Maven



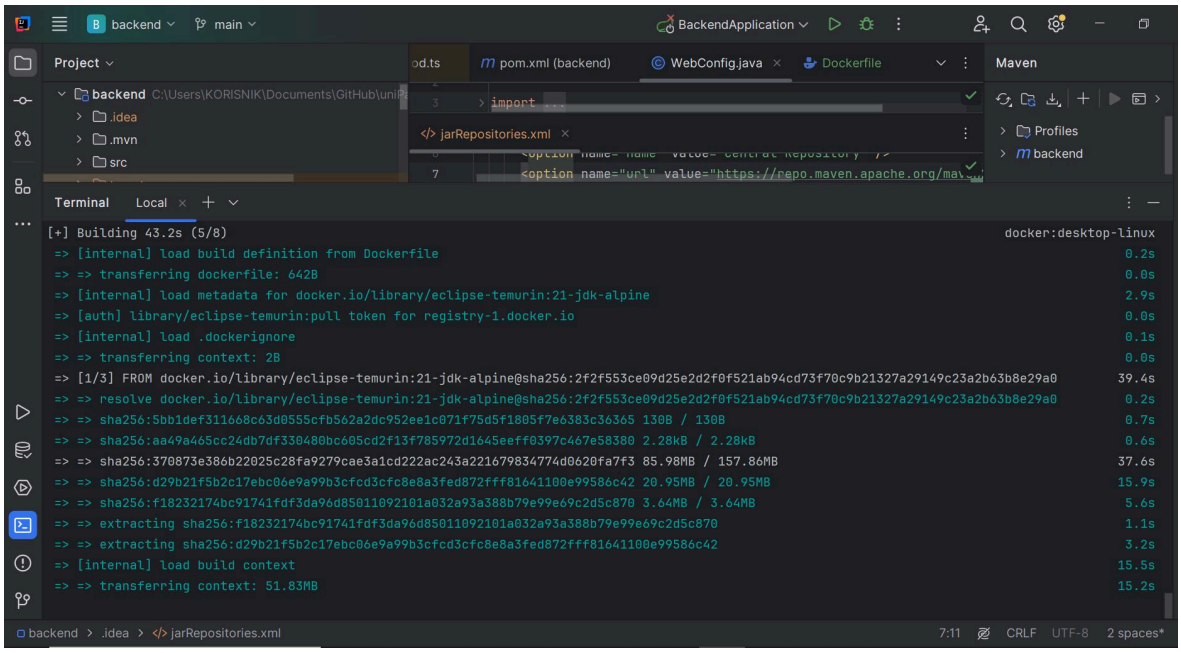
This screenshot displays the successful execution of the `mvn clean package` command, which generates a deployable JAR file in the `target/` directory. This file contains all compiled classes and dependencies needed to run the application independently. Therefore, the maven build completed with a successful JAR file output.

4.2 Verifying Application Launch from the JAR

This screenshot demonstrates the JAR being executed locally using the `java -jar` command. Spring Boot logs confirm successful initialization and port binding to `8080`.

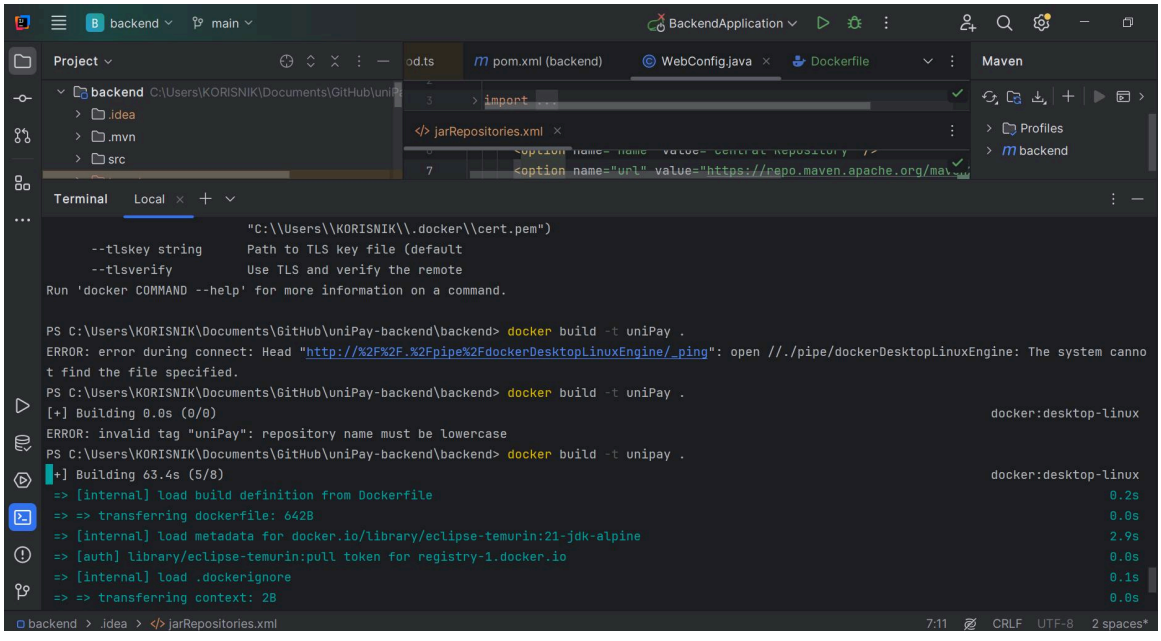


4.3 Initializing Docker Build with Dockerfile



This screenshot shows the Docker build process using a custom `Dockerfile` that packages the Spring Boot JAR into a container image based on an Alpine JDK.

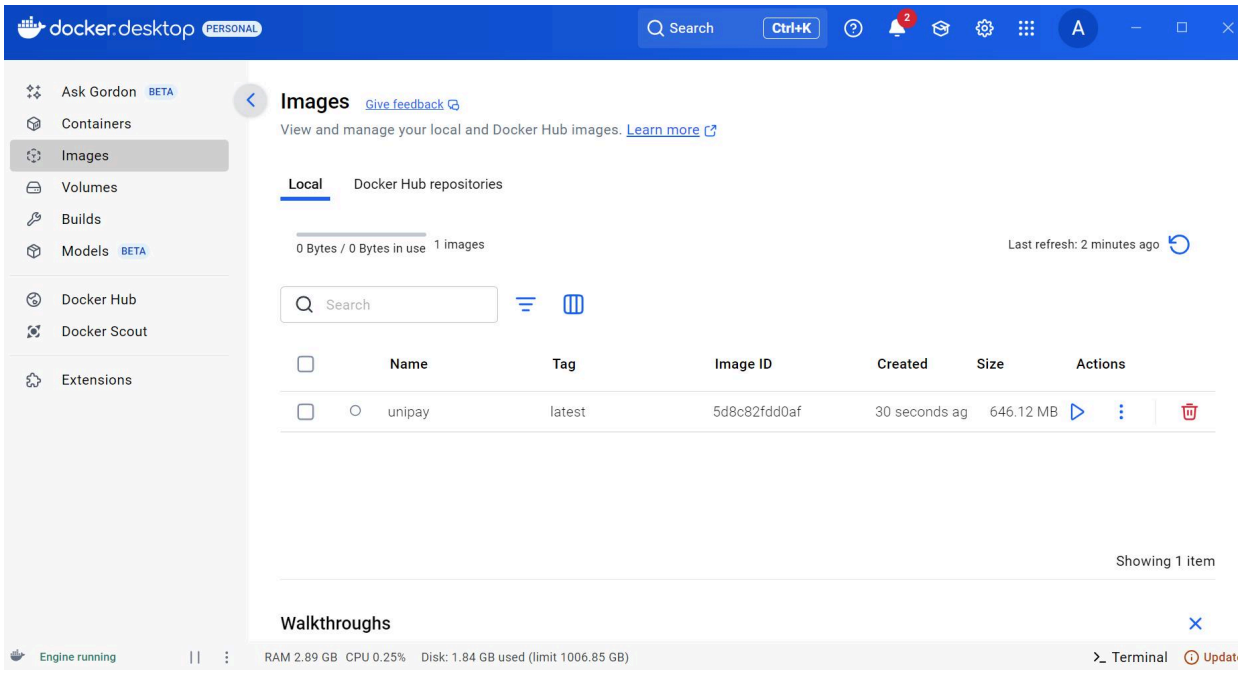
4.4 Addressing Docker Tagging and Engine Errors



This screenshot captures terminal errors resolved during the image tagging process and Docker engine availability, ensuring a clean image creation and naming convention.

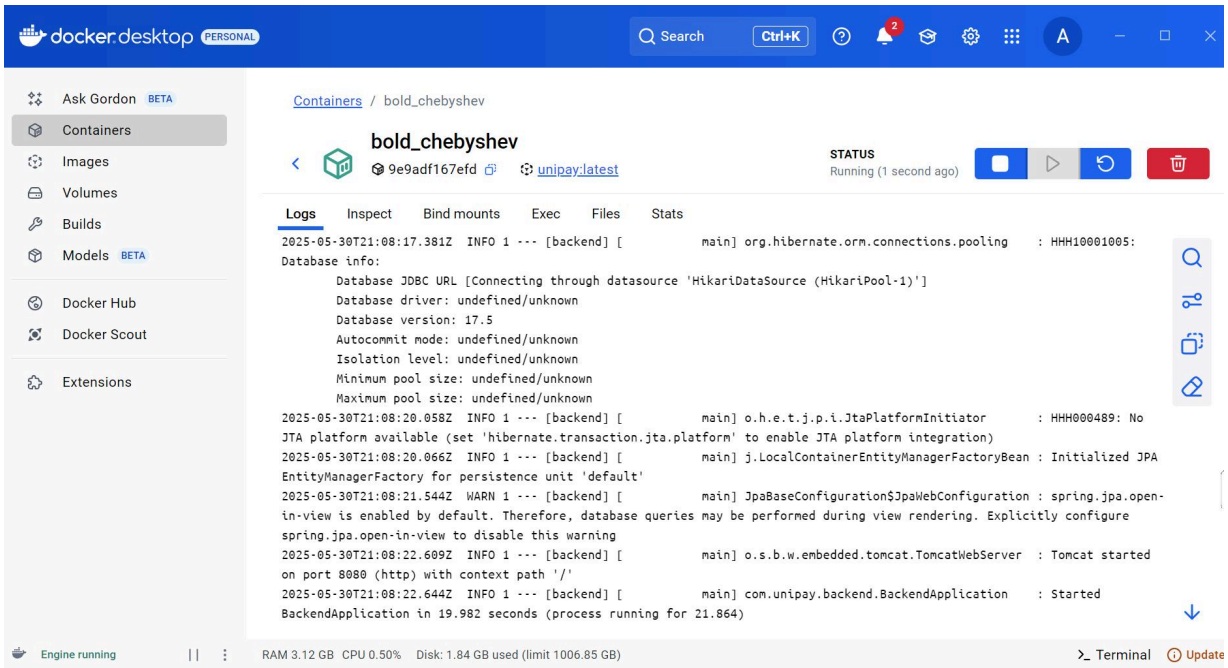
4.5 Verifying Image Build in Docker Desktop

This screenshot shows the `unipay:latest` image listed in Docker Desktop after successful local build. This confirms readiness for further deployment steps – Local image confirmed in Docker Desktop UI:

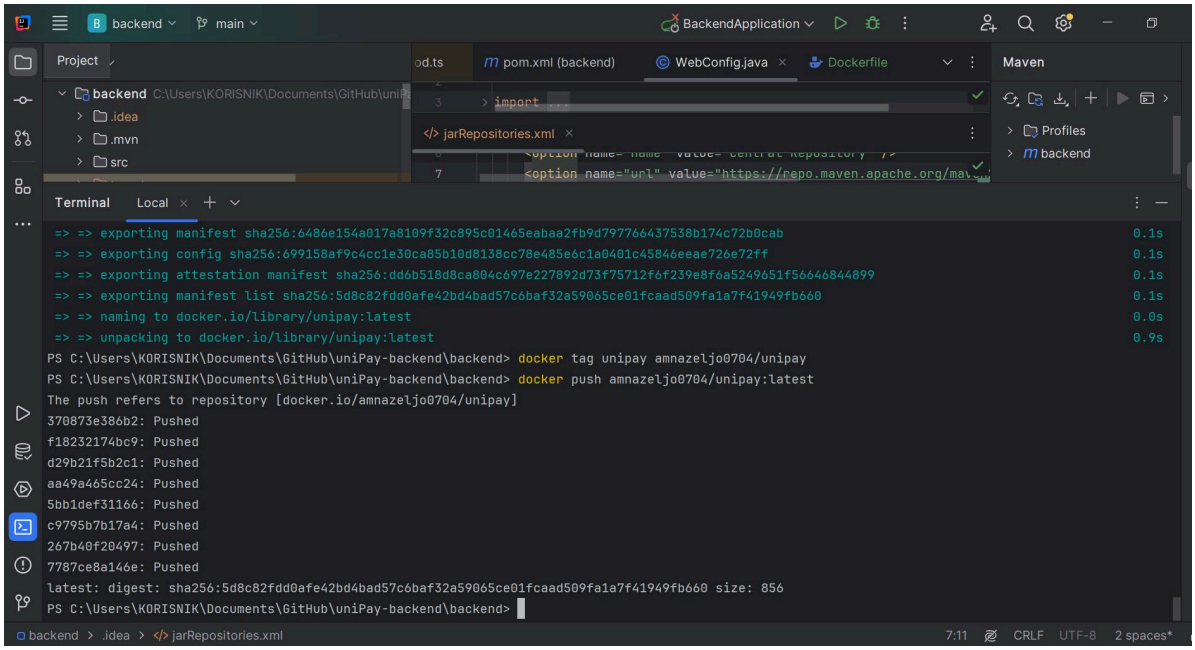


4.6 Running the Container Locally and Checking Logs

This screenshot displays a running container based on the created image. Logs indicate successful Spring Boot initialization and readiness. So, above shows the container running locally with healthy Spring Boot logs.



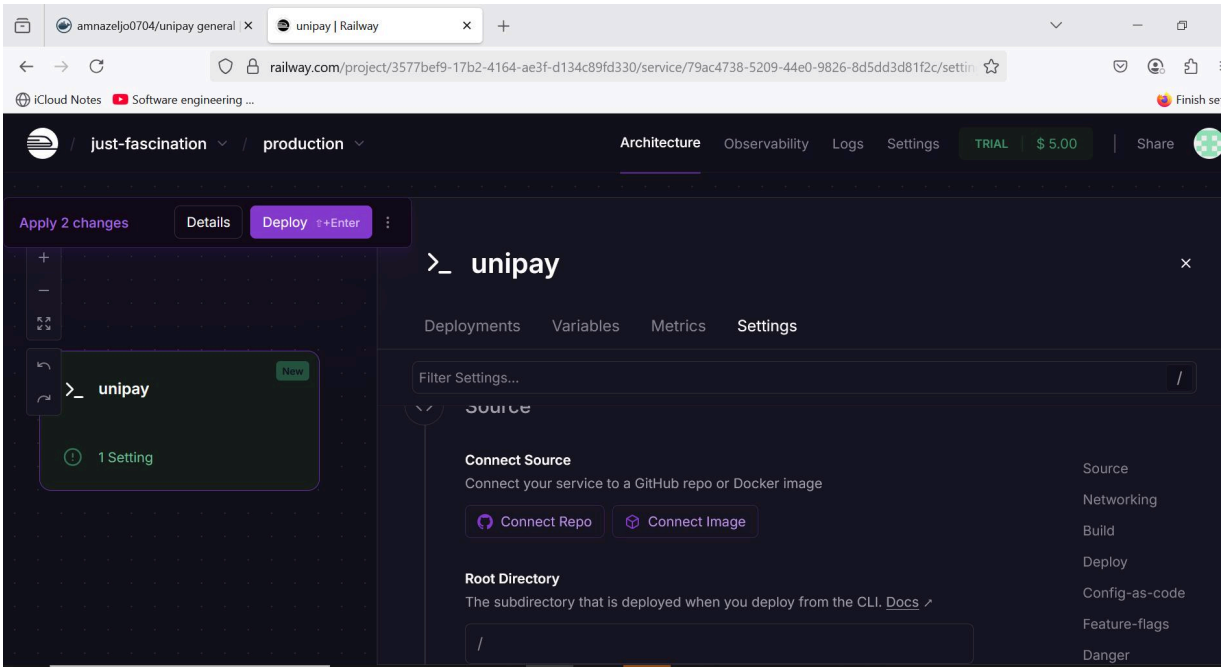
4.7 Pushing the Docker Image to DockerHub



This screenshot confirms the backend image is pushed to DockerHub registry using the tag `amnazeljo0704/unipay`. Digest information confirms the upload.

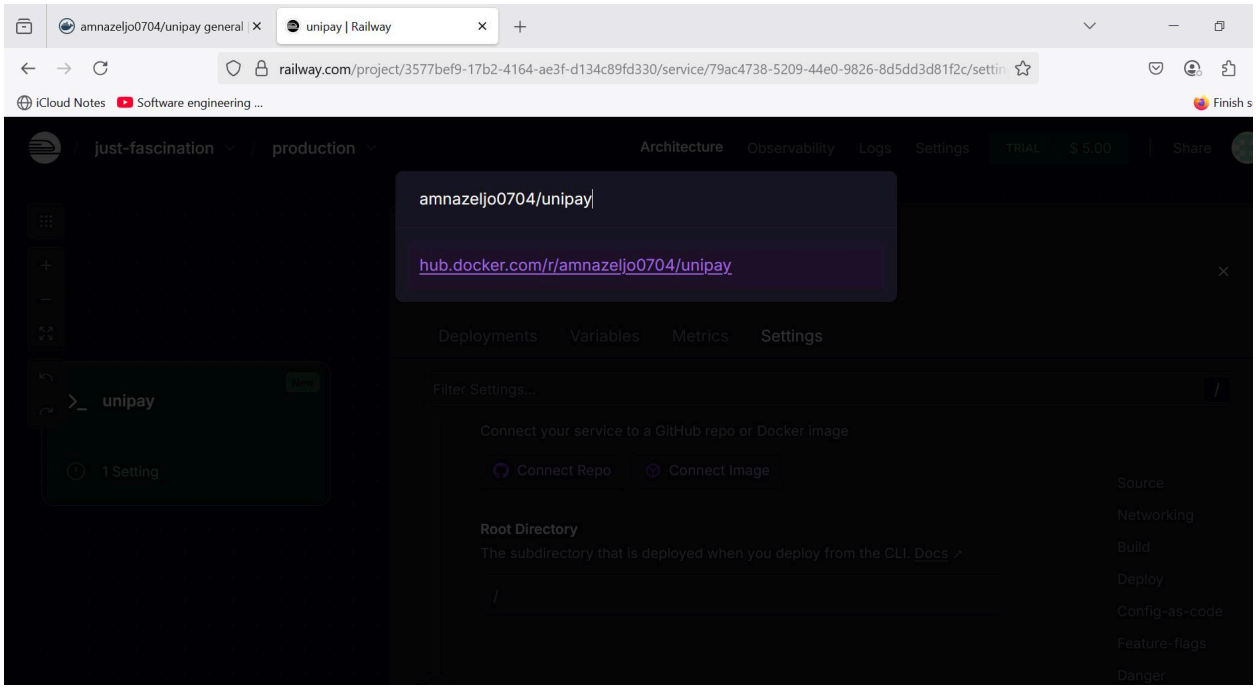
4.8 Configuring Railway Deployment from Docker Image

This screenshot shows the Railway dashboard configuration screen, where the Docker image from DockerHub was specified as the source for deployment; Railway deployment configured to pull from DockerHub:



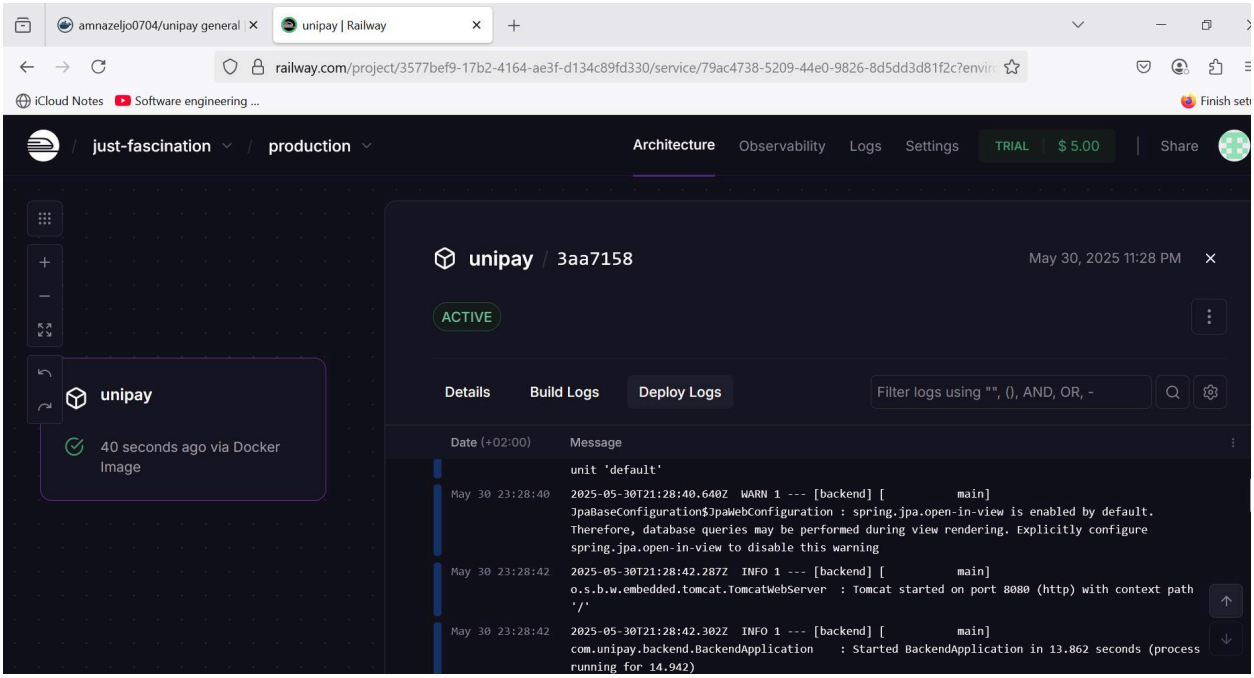
4.9 Linking Docker Image to Railway Service

This screenshot shows the linked DockerHub image URL and metadata in Railway, confirming that DockerHub is correctly integrated.



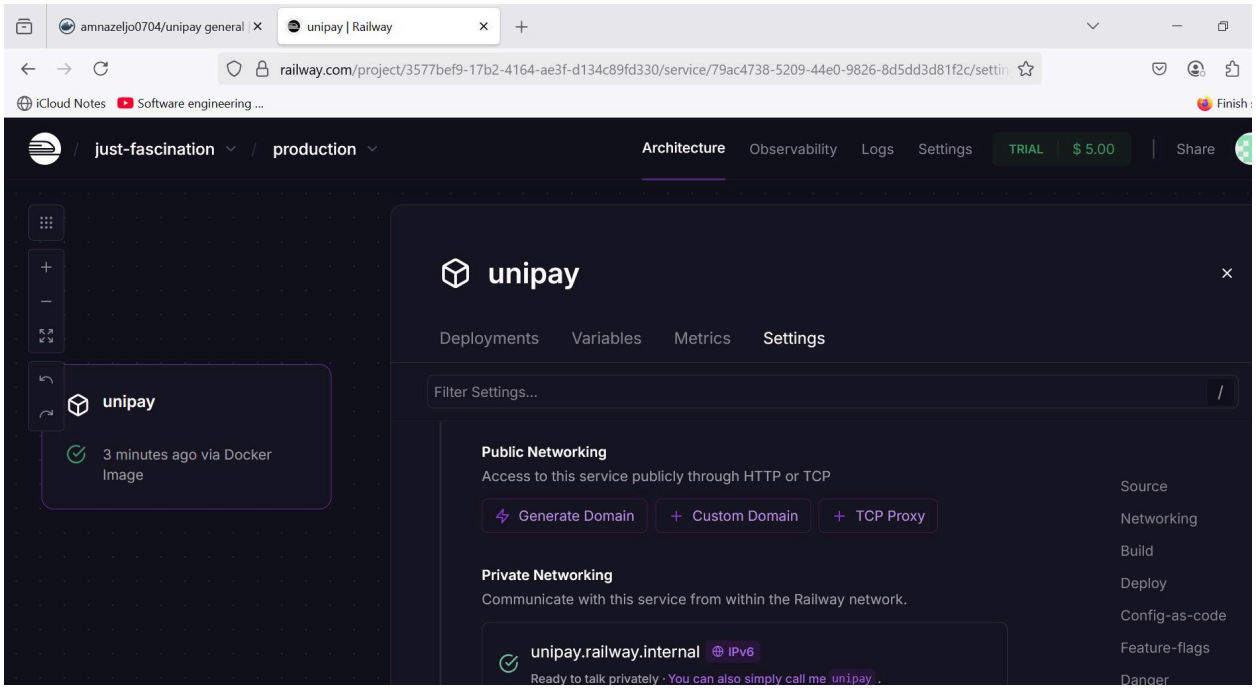
4.10 Railway Deployment Logs and Active Service

This screenshot displays successful logs from Railway after deploying the container, indicating that the backend is running successfully and accessible



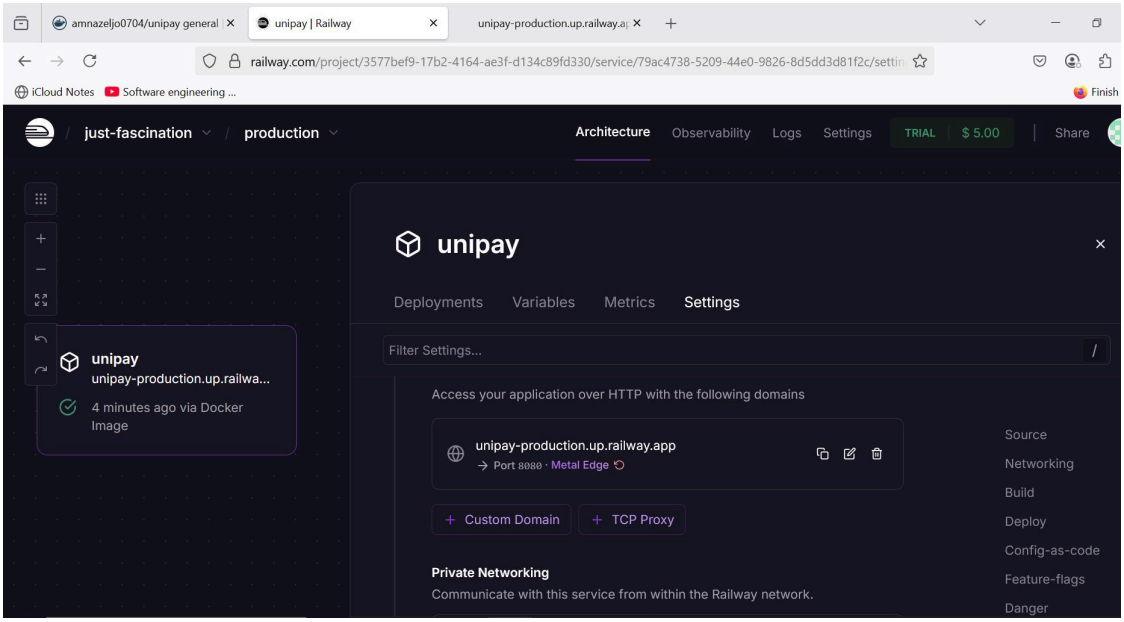
4.11. Public Networking Configuration on Railway

This screenshot shows the networking settings in the Railway platform. The Public Networking option has been enabled, allowing external HTTP access to the backend service. This step is important to expose the API endpoints to the public internet.



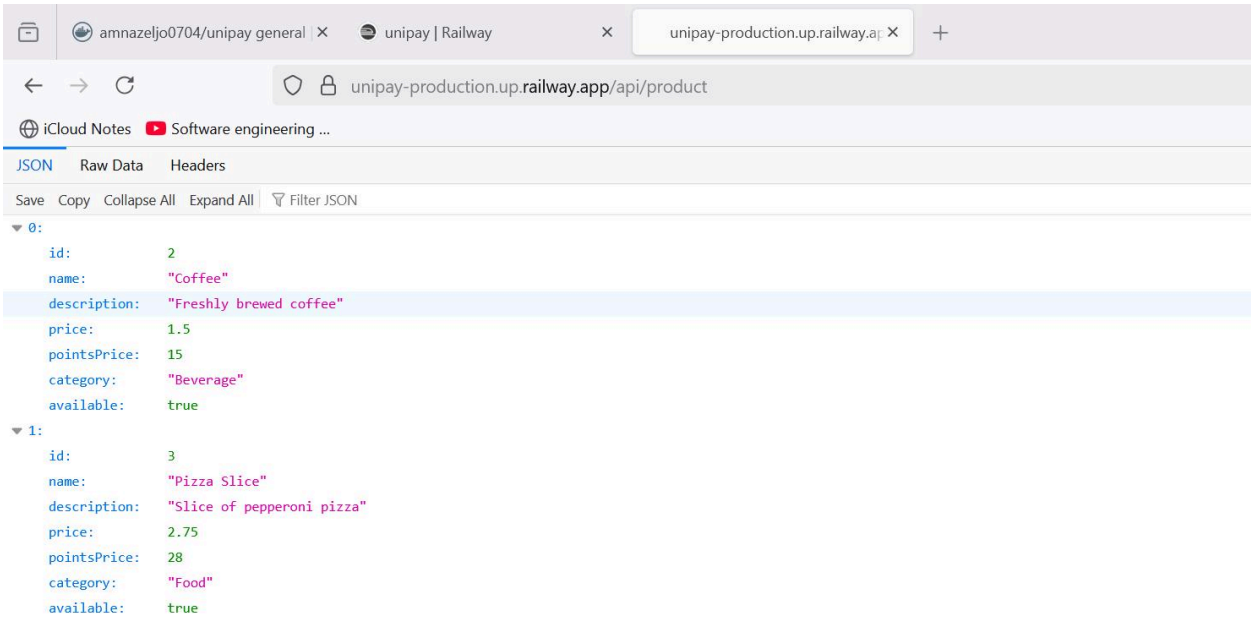
4.12. Railway Domain URL for API Access

This screenshot presents the auto-generated Railway URL: `https://unipay-production.up.railway.app`. This address services as the public entry point for accessing the production instance of the backend deployed via DockerHub and Railway.



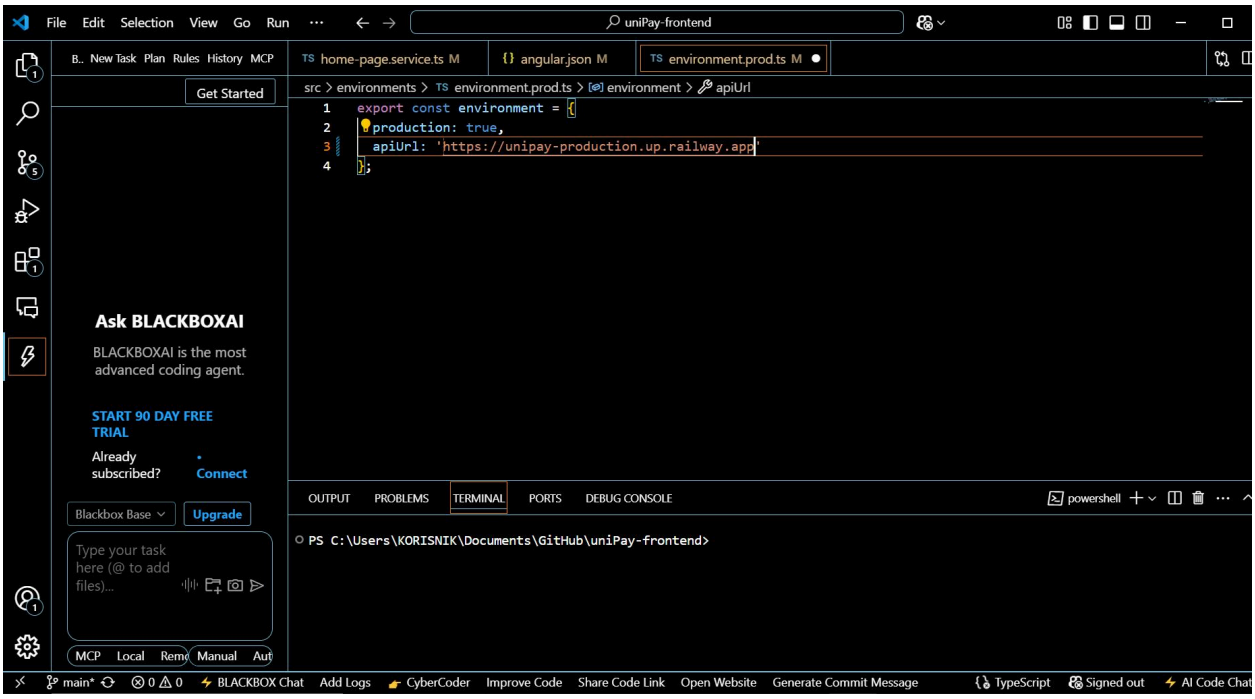
4.13. REST API Test - Verifying GET /api/product

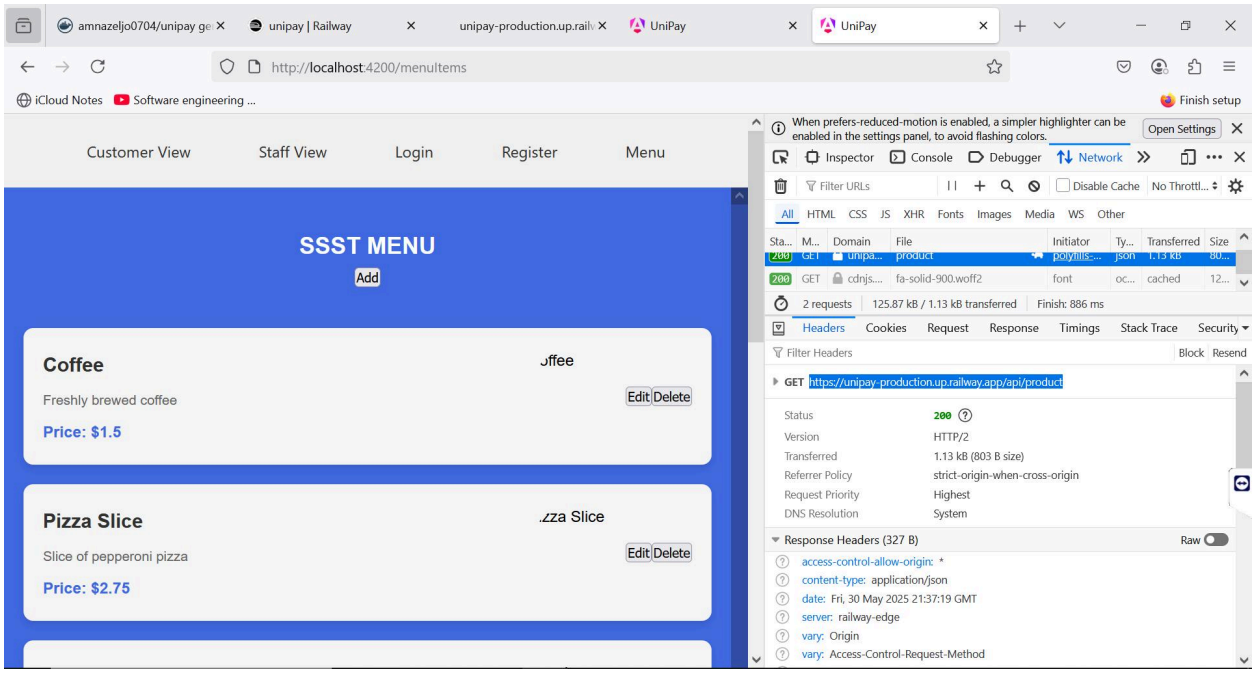
The next step was to get a successful GET request to `/api/product` endpoint using a browser. The response returns valid JSON data, confirming the backend is operational and serving correct content.



4.14. Frontend Integration via environment.prod.ts

Below shows the `environment.prod.ts` file in the Angular frontend. The `apiUrl` is correctly set to the Railway backend URL. Therefore, ensuring that the frontend connects to the deployed backend in production mode.



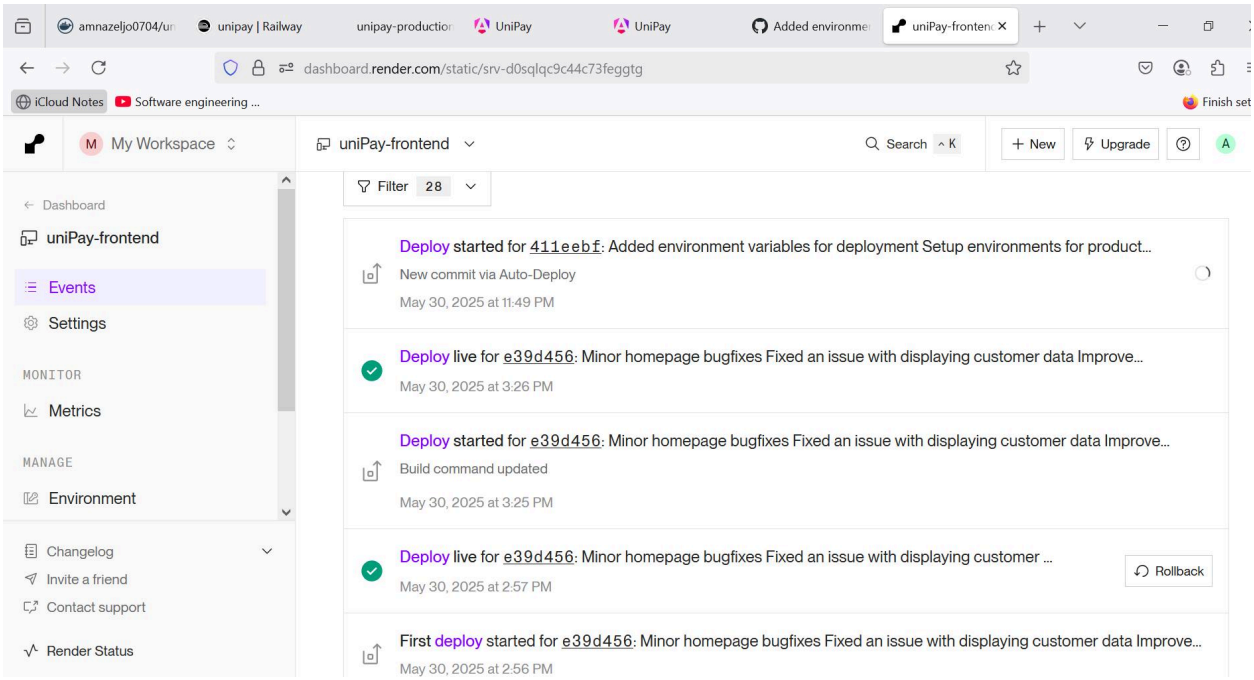


4.15. DevTools Network Tab Verification

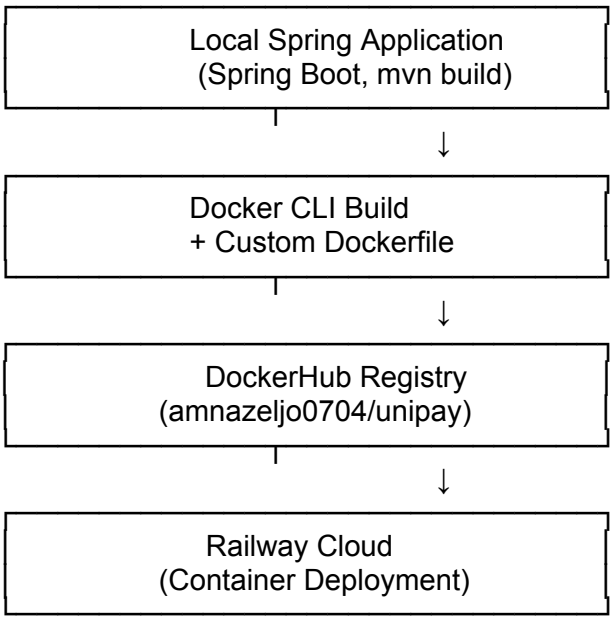
In the Angular application, Chrome DevTools shows a successful network request to `https://unipay-production.up.railway.app/api/product`. The *200 OK* status confirms that the frontend-to-backend communication works as expected in production.

The Finishing Line

Finally, the Render dashboard displays auto-deploy events triggered by GitHub commits. This confirms that the frontend is continuously deployed and consuming the live backend API endpoint configured in the environment file → Render auto-deploy events showing live frontend updates.



5. Diagrammatic Representation of the Solution



---

6. Milestones

Task	Status	Completion Date
Maven project built locally	Completed	27.05.2025
Docker image created from JAR	Completed	27.05.2025
DockerHub repository tagged and pushed	Completed	28.05.2025
Railway project configured	Completed	28.05.2025
Backend application running on Railway	Completed	29.05.2025

II. Source Code Documentation

- **Repository URL**: (private/internal Git repository)
- Structure follows standard Spring Boot conventions:
    - Controllers in `/controller`
    - Entities in `/entity`
    - DTOs and Services organized by feature
    - Configuration in `application.properties`

Dockerfile used for build includes:

```
FROM eclipse-temurin:21-jdk-alpine
COPY target/backend-0.0.1-SNAPSHOT.jar app.jar
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

III. Quality Assurance Documentation

After deployment to Railway, the following verifications were performed:

- The backend container started without runtime errors.
- Spring Boot logs confirmed application initialization.
- API endpoints were tested using Postman and responded successfully.
- No open vulnerabilities or failed service bindings.

This confirms the deployment is stable and ready for production.

IV. Tools Used for Deployment

Tool	Purpose
GitHub	Version control and CI
Maven	Build tool for Java Spring Boot project
Docker	Containerization of application
DockerHub	Registry to host backend images
Railway	Cloud deployment platform for Docker images
Visual Studio Code	Code editing and terminal access

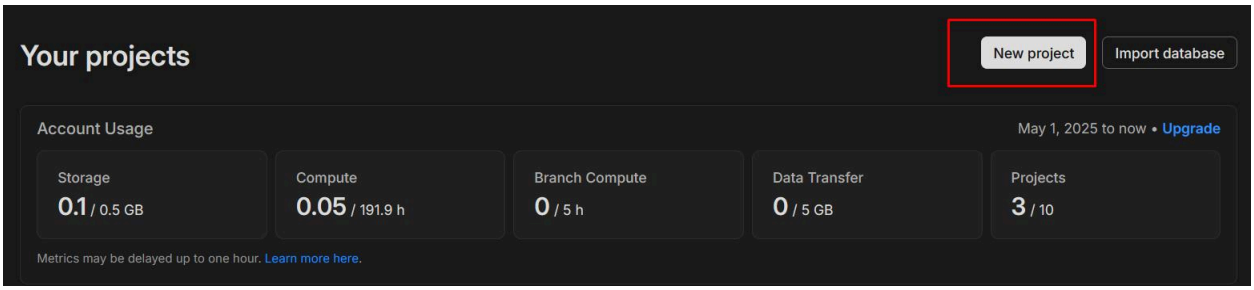
V. Conclusion

The `uniPay` backend was successfully deployed using a container-based architecture. This approach enables the application to be built and deployed reliably using Docker and Railway, ensuring future maintainability and production readiness. The process meets all the requirements for backend deployment in the course project.

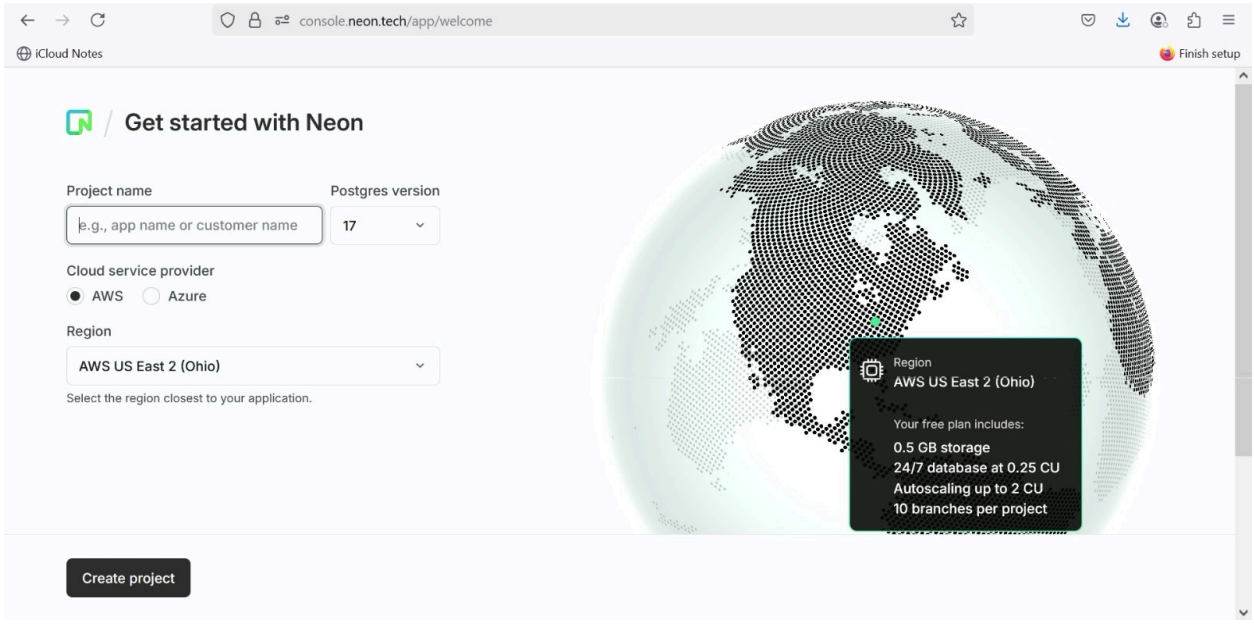
Neon PostgreSQL Database Implementation

To support the backend's data persistence layer, a PostgreSQL database was provisioned using Neon, a fully managed cloud-native PostgreSQL service.

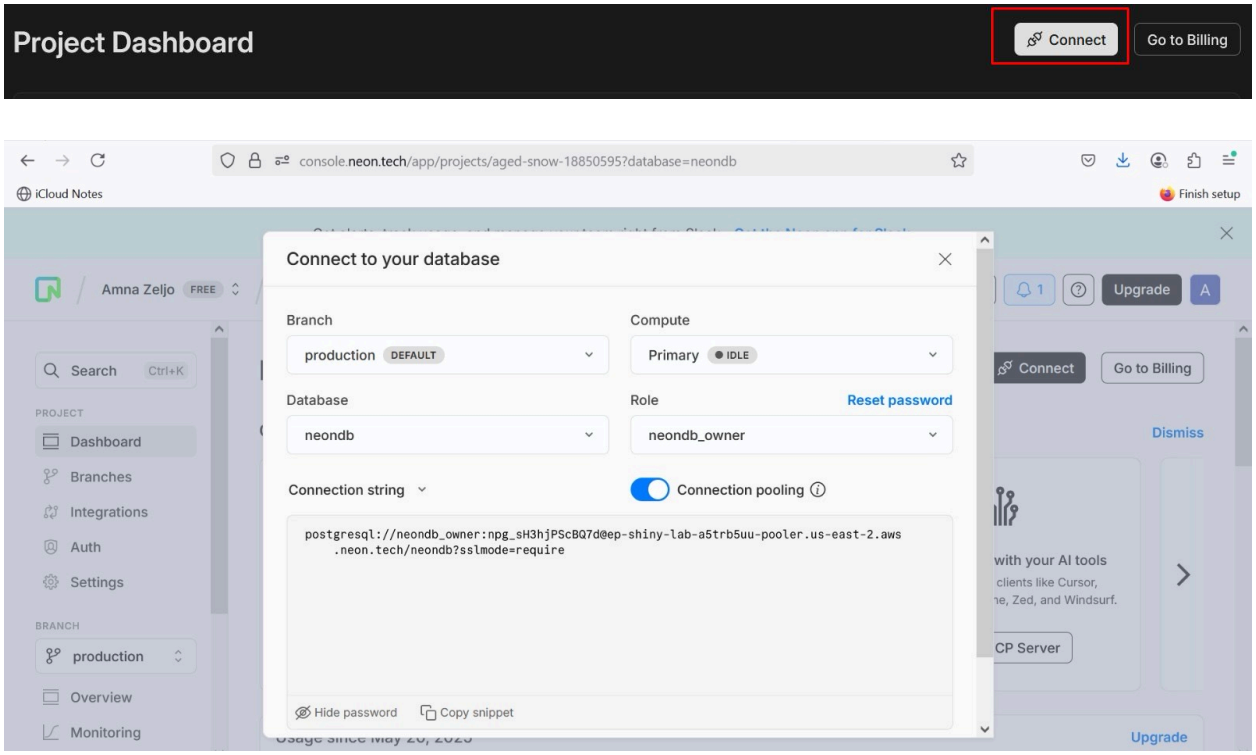
Neon was selected for its developer-friendly interface and smooth integration with modern deployment platforms such as Railway. A new project was created in the Neon dashboard, followed by the creation of a dedicated database instance, user credentials, and connection URL. This connection string was later injected as an environment variable into the Railway deployment configuration, allowing the backend to connect securely and efficiently to the database.



Neon streamlined the database setup process by automatically provisioning all necessary infrastructure components, all that was needed was for one member of the team to insert the database name.



Next, Neon handled the creation of the database instance, user roles, and connection configuration. Once the database was initialized, the provided connection credentials were used to integrate the PostgreSQL instance with the Spring Boot backend application, as shown in the following screenshot.



After, the following step shows the configuration of database connection properties in the `application.properties` file. The Spring Boot application is set up to connect to the Neon-hosted PostgreSQL instance using the provided JDBC URL, credentials, and driver class.

